

Micropolar Elasticity: Theory and FEniCS Implementation

Aditya Sanjeev Badgular baditya@umich.edu

Department of Mechanical Engineering, University of Michigan

Abstract

This report details the development of an open-source computational framework for simulating micropolar elasticity using the FEniCS finite element library. The implementation is based on a theoretical foundation that accounts for size-dependent effects in materials with microstructure, allowing for both elastic and damaged behavior to be accurately represented. By utilizing FEniCS's automated code generation, the computational procedures are closely aligned with the mathematical model, ensuring clarity and efficiency. The report covers the construction of a mixed finite element formulation for plane micropolar elasticity and validates the approach through comparison with analytical benchmarks. The open-source nature of the code is intended to support reproducibility, third-party verification, and further advancements in modeling higher-order continuum mechanics for materials that exhibit size effects.

1 Introduction

The mechanical reactions of materials with underlying microstructural features such as foams, granular assemblies, engineered metamaterials, and different geomaterials frequently rely on their distinctive length scales. These size effects, which occur in both elastic and inelastic regimes, cannot be fully represented by traditional constitutive models or first-order homogenization procedures because they fail to capture the requisite higher-order kinematics.

Phase field and other nonlocal fracture modeling techniques provide a straightforward way to add a length scale into damage mechanics by depicting cracks as smoothly shifting fields rather than explicit discontinuities. This regularization allows for the simulation of intricate crack evolution, including branching and coalescence.

Recent theoretical advances have addressed these issues by developing models that contain micropolar elasticity, integrating size effects directly via higher-order continuum mechanics. This approach decouples the regularization length from the inherent material length scales, resulting in a more physically relevant framework for micro structured materials.

This report provides a detailed explanation of the mixed finite element implementation for micropolar elasticity utilizing FEniCS, numerically testing on a problem and validating with an analytical solution.

2 Implementation of Micropolar Elasticity in FEniCS

2.1 Problem Statement and Kinematics

Consider a micropolar elastic body occupying a region $\mathcal{B} \subset \mathbb{R}^2$. Each material point $\mathcal{P}$ within this body is identified by a position vector $\mathbf{x} \in \mathcal{B}[1]$. In the context of infinitesimal deformation, the motion of a material point is described by two independent quantities:

Translational Displacement: The vector field $\mathbf{u}(\mathbf{x}, t)$ represents the displacement of the material point $\mathbf{x}$ at time t . This is analogous to the displacement field in classical elasticity.

Micro-rotation: The vector field $\boldsymbol{\theta}(\mathbf{x}, t)$ represents the local rotation of the material point $\mathbf{x}$ at time t . This micro-rotation is a fundamental characteristic of micropolar materials and

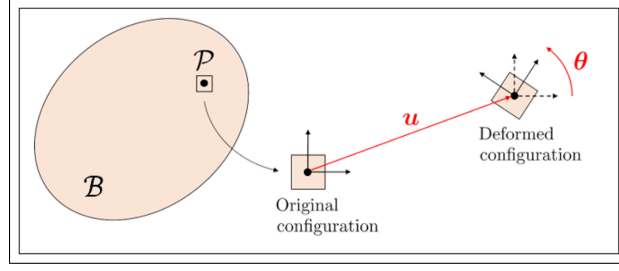


Figure 1: Kinematics of a micropolar continuum [2]

distinguishes them from classical materials. It is important to emphasize that θ is independent of the rotational part of the polar decomposition of the displacement gradient (i.e., macro-rotation).

Micropolar Strain (ε): This tensor measures the deformation of the material, taking into account both the translational displacement and the micro-rotation. The strain tensor ε is defined as:

$$\varepsilon = \nabla u + \epsilon \theta_3$$

Here, ϵ_{ijk} is the Levi-Civita permutation tensor.

Micro-Curvature (κ): This tensor measures the relative rotation between neighboring material points. It is defined as:

$$\varphi = \begin{bmatrix} \frac{\partial \theta_3}{\partial x_1} \\ \frac{\partial \theta_3}{\partial x_2} \end{bmatrix}$$

It captures the bending and twisting aspects of the deformation that are not present in classical elasticity. Since the micropolar strain tensor is non-symmetric, we therefore decompose and, the micropolar strain tensor into symmetric (ε^{sym}) and skew-symmetric parts (ε^{skew})

$$\varepsilon^{sym} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T); \quad \varepsilon^{skew} = \frac{1}{2}(\nabla \mathbf{u} - \nabla \mathbf{u}^T) + \epsilon \theta_3$$

After substituting the respective quantities in the above equations we obtain the following relations:

$$\varepsilon^{sym} = \begin{bmatrix} \frac{\partial u_1}{\partial x_1} & \frac{1}{2} \left(\frac{\partial u_1}{\partial x_2} + \frac{\partial u_2}{\partial x_1} \right) \\ \frac{1}{2} \left(\frac{\partial u_2}{\partial x_1} + \frac{\partial u_1}{\partial x_2} \right) & \frac{\partial u_2}{\partial x_2} \end{bmatrix}; \quad \varepsilon^{skew} = \begin{bmatrix} 0 & \frac{1}{2} \left(\frac{\partial u_1}{\partial x_2} - \frac{\partial u_2}{\partial x_1} \right) + \theta_3 \\ \frac{1}{2} \left(\frac{\partial u_2}{\partial x_1} - \frac{\partial u_1}{\partial x_2} \right) - \theta_3 & 0 \end{bmatrix}$$

Note: ε^{sym} is equivalent to the Boltzmann strain tensor in the classical non-polar approach.

2.2 Constitutive model and strain energy split

To achieve reliable elastic responses, the micropolar strain energy must meet the strong ellipticity criterion. To meet this condition, we use a quadratic micropolar strain energy density as follows,

$$\psi_e(\varepsilon, \varphi) = \frac{1}{2} \varepsilon : \mathbb{C} : \varepsilon + \frac{1}{2} \varphi : \mathbb{D} : \varphi$$

$\mathbb{C}$ and $\mathbb{D}$ are constitutive moduli of the fourth-order tensor and have major symmetries; which means that, $\mathbb{C} = C_{ijkl} = C_{klij}$ and $\mathbb{D} = D_{ijkl} = D_{klij}$

In circumstances when elastic moduli $\mathbb{C}$ and $\mathbb{D}$ are positive definite, the elastic body requires work to deform from its unloaded equilibrium configuration. We assume that the material is isotropic and define the elastic moduli $\mathbb{C}$ and $\mathbb{D}$ as follows:

$$\mathbb{C} = \lambda(\mathbf{I} \otimes \mathbf{I}) + (\mu + \kappa)\mathbb{I} + \mu\bar{\mathbb{I}}; \quad \mathbb{D} = \alpha(\mathbf{I} \otimes \mathbf{I}) + \beta\bar{\mathbb{I}} + \gamma\mathbb{I}.$$

λ , μ , κ , α , β , and γ are the material constants. The substantial ellipticity of the strain energy density requires the following inequalities to hold [2].

$$3\lambda + 2\mu + \kappa \geq 0; \quad 2\mu + \kappa \geq 0; \quad \kappa \geq 0; \quad 3\alpha + \beta + \gamma \geq 0; \quad \gamma + \beta \geq 0; \quad \gamma - \beta \geq 0.$$

2.2.1 Strain energy decomposition

To investigate the relationship between size-dependent micropolar elasticity and fracture mechanisms, the strain energy density is divided into three parts: the Boltzmann part, the micro continuum coupling part, and the pure micro rotational part [2].

$$\psi_e(\boldsymbol{\varepsilon}, \boldsymbol{\varphi}) = \psi_e^B(\boldsymbol{\varepsilon}^{sym}) + \psi_e^C(\boldsymbol{\varepsilon}^{skew}) + \psi_e^R(\boldsymbol{\varphi})$$

The partitioned strain energy densities can be expressed as,

$$\psi_e^B(\boldsymbol{\varepsilon}^{sym}) = \frac{1}{2}[\lambda(\boldsymbol{\varepsilon}^{sym} : \mathbf{I})^2 + (2\mu + \kappa)\boldsymbol{\varepsilon}^{sym} : \boldsymbol{\varepsilon}^{sym}]$$

$$\psi_e^C(\boldsymbol{\varepsilon}^{skew}) = \frac{1}{2}\kappa\boldsymbol{\varepsilon}^{skew} : \boldsymbol{\varepsilon}^{skew}$$

$$\psi_e^R(\boldsymbol{\varphi}) = \frac{1}{2}[\alpha(\boldsymbol{\varphi} : \mathbf{I})^2 + \beta\boldsymbol{\varphi} : \boldsymbol{\varphi}^T + \gamma\boldsymbol{\varphi} : \boldsymbol{\varphi}]$$

In classical elasticity, the strain energy density function, which quantifies the energy stored within a deformed material, is solely dependent on the symmetric strain tensor. This tensor captures the deformation resulting from the displacement of material points. However, micropolar elasticity introduces a more nuanced perspective by considering the independent micro-rotation of these material points (Here, $\boldsymbol{\varepsilon}^{skew}$), represented by the micro-rotation vector. This fundamental difference necessitates a modification of the strain energy density formulation.

2.2.2 Force Stress

In micropolar elasticity, force stress refers to the classical stress tensor but with a skew-symmetric stress component, generally represented as, which results from the translational displacement of material points. This is similar to the stress concept in classical elasticity and indicates the force per unit area.

$$\boldsymbol{\sigma} = \frac{\partial \psi_e}{\partial \boldsymbol{\varepsilon}} = \frac{\partial}{\partial \boldsymbol{\varepsilon}}(\psi_e^B + \psi_e^C) = \lambda(\boldsymbol{\varepsilon}^{sym} : \mathbf{I})\mathbf{I} + (2\mu + \kappa)\boldsymbol{\varepsilon}^{sym} + \kappa\boldsymbol{\varepsilon}^{skew}$$

The above equation states that the force stress tensor $\boldsymbol{\sigma}$ is derived from the strain energy density function by taking its partial derivative with respect to the micropolar strain tensor. The equation also shows a linear relationship between force stress and micropolar strain through the fourth-order elasticity tensor $\mathbf{C}$. This is a generalized form of Hooke's law, adapted for micropolar materials. In essence, force stress, in this context, represents the part of the stress that is related to the strain, similar to classical elasticity.

The force stress can also be partitioned into the Boltzmann part $\boldsymbol{\sigma}^b$ and the microcontinuum coupling part $\boldsymbol{\sigma}^c$, which are the results of pure non-polar deformation and the microcontinuum coupling effects, respectively.

$$\boldsymbol{\sigma}^b = \lambda(\boldsymbol{\varepsilon}^{sym} : \mathbf{I})\mathbf{I} + (2\mu + \kappa)\boldsymbol{\varepsilon}^{sym}; \quad \boldsymbol{\sigma}^c = \kappa\boldsymbol{\varepsilon}^{skew}$$

2.2.3 Couple Stress

The couple stress induced by pure micro-rotation at the material points, represented as moment per unit area can be calculated as follows.

$$\mathbf{m}^R = \frac{\partial \psi_e}{\partial \boldsymbol{\varphi}} = \frac{\partial \psi_e^R}{\partial \boldsymbol{\varphi}} = \alpha(\boldsymbol{\varphi} : \mathbf{I})\mathbf{I} + \beta \boldsymbol{\varphi}^T + \gamma \boldsymbol{\varphi}$$

2.3 Balance Laws

The balance of linear momentum and the balance of angular momentum govern the behavior of the micropolar continuum.

The equation for balance of linear momentum is given by:

$$\operatorname{div} \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0}$$

assuming body forces $\mathbf{b}$ as 0, we get,

$$\operatorname{div} \left[\lambda(\boldsymbol{\varepsilon}^{sym} : \mathbf{I})\mathbf{I} + (2\mu + \kappa)\boldsymbol{\varepsilon}^{sym} + \kappa(\boldsymbol{\varepsilon}^{skew})^T \right] = \mathbf{0}$$

The equation for balance of angular momentum is given by:

$$\begin{aligned} \operatorname{div} \mathbf{m}^R + \epsilon_{ijk} : (\boldsymbol{\sigma}^C)^T &= \mathbf{0} \\ \operatorname{div} [\alpha(\boldsymbol{\varphi} : \mathbf{I})\mathbf{I} + \beta \boldsymbol{\varphi}^T + \gamma \boldsymbol{\varphi}] + \epsilon : \kappa(\boldsymbol{\varepsilon}^{skew})^T &= \mathbf{0} \end{aligned}$$

2.4 Variational formulation

This approach is a cornerstone of continuum mechanics, providing an alternative to the direct solution of differential equations. Instead of solving the strong form of the governing equations, which involves pointwise satisfaction of the equations, the variational method seeks solutions that satisfy an integral form of the equations.

Weak Form: The variational formulation leads to the weak form of the governing equations. This involves formulating the problem in terms of integral equations, where the solution is required to satisfy the equations in an average sense over the domain of the problem. The weak form is obtained by multiplying the governing differential equations by suitable test functions and integrating over the domain.

Boundary Conditions: Boundary conditions are incorporated into the variational formulation through the integral terms. This allows for a more natural treatment of both essential boundary conditions, that is, Dirichlet (displacement $\partial \mathcal{B}_u$ and micro-rotation $\partial \mathcal{B}_\theta$) Neumann (traction $\partial \mathcal{B}_{t_\sigma}$ and moment $\partial \mathcal{B}_{t_m}$).

Test Functions: The choice of test functions is crucial in the variational formulation. These functions belong to a function space and are used to "test" the governing equations. The solution to the problem is then sought within a space of functions that satisfy the weak form for all admissible test functions.

The trial function spaces for the displacement ($\mathbf{u}$) and microrotation ($\boldsymbol{\theta}$) are defined using Taylor-Hood elements to ensure stability. Find the trial functions below,

$$\begin{aligned} V_u &= \{ \mathbf{u} : \mathcal{B} \rightarrow \mathbb{R}^2 \mid \mathbf{u}|_{\partial \mathcal{B}_u} = \hat{\mathbf{u}} \} & V_\eta &= \{ \boldsymbol{\eta} : \mathcal{B} \rightarrow \mathbb{R}^2 \mid \boldsymbol{\eta}|_{\partial \mathcal{B}_u} = \mathbf{0} \} \\ V_\theta &= \{ \boldsymbol{\theta} : \mathcal{B} \rightarrow \mathbb{R} \mid \boldsymbol{\theta}|_{\partial \mathcal{B}_\theta} = \hat{\boldsymbol{\theta}} \} & V_\xi &= \{ \boldsymbol{\xi} : \mathcal{B} \rightarrow \mathbb{R} \mid \boldsymbol{\xi}|_{\partial \mathcal{B}_\theta} = \mathbf{0} \} \end{aligned}$$

These test functions are multiplied with the governing equations and are further integrated over the domain of the problem, incorporating the boundary conditions of the problem into the

integral equations. Then the standard weighted residual procedure yields the weak statement for both linear and angular balance equations as below.

$$\int_{\mathcal{B}} \varepsilon(\boldsymbol{\eta}, \boldsymbol{\xi}) : \boldsymbol{\sigma}(\mathbf{u}, \boldsymbol{\theta}_3) dV = 0$$

$$\int_{\mathcal{B}} \boldsymbol{\varphi}(\boldsymbol{\xi}) \cdot \mathbf{m}^R(\boldsymbol{\theta}_3) dV - \int_{\mathcal{B}} \boldsymbol{\xi} \cdot \boldsymbol{\epsilon} : \boldsymbol{\sigma}^C(\mathbf{u}, \boldsymbol{\theta}_3) dV = 0$$

2.5 FEniCS Implementation

The variational form is implemented in FEniCS using the Unified Form Language (UFL). For example, the micropolar strain is stated in the below code block [1]:

```
def epsilon(u, theta):
    return as_tensor([[u[0].dx(0), u[1].dx(0) - theta],
                      [u[0].dx(1) + theta, u[1].dx(1)]])
```

The weak form of the governing equations is then defined using these quantities.

```
a = inner(epsilon(eta, xi), sigma(u, theta)) * dx \
    + inner(phi(xi), m_R(theta)) * dx \
    - inner(xi, E3_sigma_C(u, theta)) * dx
L = dot(T, eta) * ds
```

Where **eta** and **xi** are the test functions for displacement and microrotation, respectively, and **T** represents the applied traction. The boundary conditions are applied using FEniCS's **DirichletBC** functionality. The linear system is solved using FEniCS's linear variational solver.

3 Validation of Micropolar theory on a plate with hole

We consider a classical problem of an infinite plate with a hole in fracture mechanics depicted in Figure 2 for which an analytical solution exists, allowing for a direct comparison between the numerical results obtained from FEniCS and the theoretical predictions. The circular hole introduces a stress concentration, meaning that the stress around the hole is much higher than the applied stress. This makes it a good test case for fracture models, as cracks often initiate at stress concentrations. In this study a rectangular plate under pure tension σ_0 that has a circular hole with diameter D at the center is considered.

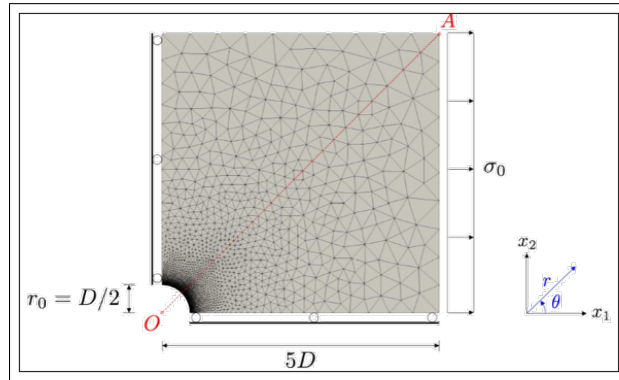


Figure 2: Quarter-plate with a circular hole

Regarding the boundary conditions, the left and bottom edges are supported by fixed micro-rotation rollers, a uniform tension σ_0 on the right boundary is applied. All other boundaries

are maintained traction and moment free during the simulation. Considering the following material parameters for this example, shear modulus $G = 50$ GPa, Poisson ratio $\nu = 0.3$, and bending characteristic length $l = D$. Within this problem domain, we investigate the effect of the micropolar effect on stress concentration factors (SCF) (that is, the ratio between the highest force stress to the applied stress σ_0). Figure 3 illustrates the distribution of normalized stress fields in the domain with a coupling number of $N = 0.9$.

Table 1 compares the numerically calculated stress concentration factor (SCF_{FEM}) to the analytical solution ($SCF_{Analytical}$) presented in Eringen (2012)[3] (Matlab Code for calculations can be found in Appendix 2). We define the relative error (e) as follows:

$$e = \frac{|SCF_{FEM} - SCF_{Analytical}|}{SCF_{Analytical}}$$

Table 1: Comparison of SCF values

Result	N=0.9
Numerical result, SCF_{FEM}	2.036
Analytical solution, $SCF_{analytical}$	2.38
Relative error, e	14.45%

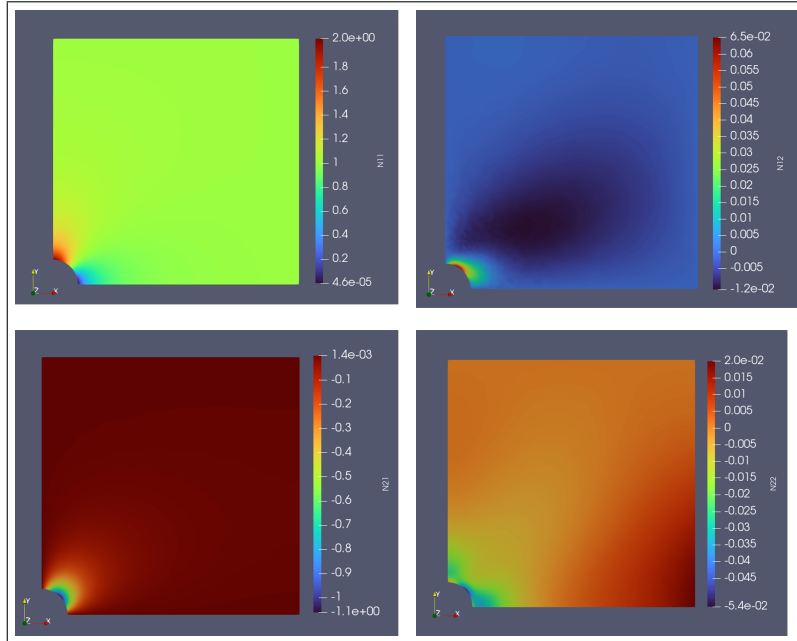


Figure 3: Normalized force stress fields with $N = 0.9$

4 Conclusion

The open-source implementation of a micropolar phase field fracture model in FEniCS offers a valuable tool for simulating cohesive fracture in micropolar continua. The finite element code (Can be found in Appendix 1), designed with a close correspondence to the underlying mathematical formulation, enables researchers and practitioners to explore the complex behavior of these materials. Preliminary verification exercises, such as the analysis of stress concentration around a circular hole, demonstrate the code's ability to reproduce known analytical solutions, thus validating its accuracy. This implementation provides a foundation for Micropolar Elasticity Theory, allowing for future study on Micropolar Phase Field Fracture and the further development of higher-order continuum damage models.

5 References

- [1] Suh, H. S., and Sun, W., 2019, *AN OPEN-SOURCE FENICS IMPLEMENTATION OF A PHASE FIELD FRACTURE MODEL FOR MICROPOLAR CONTINUA*.
- [2] Suh, H. S., Sun, W. C., and O'Connor, D. T., 2020, "A Phase Field Model for Cohesive Fracture in Micropolar Continua," *Computational Methods Appl Mech Eng*, 369.
- [3] *Microcontinuum Field Theories*.

Appendix 1: FEniCS Code

```
1 # -----
2 # FEniCS implementation: Micropolar elasticity
3 # Written by: Hyoung Suk Suh (h.suh@columbia.edu)
4 # -----
5
6 from dolfin import *
7 import sys
8 import time
9
10
11 tic = time.time()
12
13 # -----
14 # Input parameters
15 # -----
16 # Mesh and result file names
17 file_name = 'plate_with_a_hole' # Input/output directory name
18
19 # Material parameters (micropolar elasticity)
20 G = 50.0e9 # Shear modulus [Pa]
21 nu = 0.3 # Poisson's ratio
22 l = 2.0 # Characteristic length [m]
23 N = 0.9 # Coupling number
24
25 # Applied traction
26 trac = 1.0e3 # traction [Pa]
27
28 # -----
29 # Read mesh
30 # -----
31 mesh = Mesh('./'+file_name+'/' + file_name + '.xml')
32
33 dim = mesh.geometric_dimension()
34 mesh_coord = mesh.coordinates()
35 mesh_xmin = min(mesh_coord[:,0])
36 mesh_xmax = max(mesh_coord[:,0])
37 mesh_ymin = min(mesh_coord[:,1])
38 mesh_ymax = max(mesh_coord[:,1])
39 # -----
40 # Define function spaces
41 # -----
42 u_elem = VectorElement('CG', mesh.ufl_cell(), 2) # displacement
43 theta_elem = FiniteElement('CG', mesh.ufl_cell(), 1) # rotation
44
45 mixedUT = u_elem*theta_elem
46
47 V = FunctionSpace(mesh, mixedUT)
48
49 U, T = V.split()
50 U_0, U_1 = U.split() # 2 dofs for disp. (U_0, U_1) / 1 dof for rotation (T)
51 # -----
52 # Define boundary conditions
53 # -----
54 top = CompiledSubDomain("near(x[1], mesh_ymax) && on_boundary",
55                          mesh_ymax = mesh_ymax)
56 bottom = CompiledSubDomain("near(x[1], mesh_ymin) && on_boundary",
57                             mesh_ymin = mesh_ymin)
58 left = CompiledSubDomain("near(x[0], mesh_xmin) && on_boundary",
59                           mesh_xmin = mesh_xmin)
60 right = CompiledSubDomain("near(x[0], mesh_xmax) && on_boundary",
61                            mesh_xmax = mesh_xmax)
```



```

62
63 # Dirichlet boundary
64 BC_left      = DirichletBC(U_0, Constant(0.0), left)
65 BC_bottom    = DirichletBC(U_1, Constant(0.0), bottom)
66
67 BC_left_theta = DirichletBC(T, Constant(0.0), left)
68 BC_bottom_theta = DirichletBC(T, Constant(0.0), bottom)
69
70 BC = [BC_left, BC_bottom, BC_left_theta, BC_bottom_theta]
71
72 # Neumann boundary
73 boundaries = MeshFunction('size_t', mesh, dim-1)
74 boundaries.set_all(0)
75 right.mark(boundaries, 1)
76 traction = Constant((trac, 0.0))
77
78 ds = Measure("ds")(subdomain_data=boundaries)
79 n   = FacetNormal(mesh)
80
81 # -----
82 # Define variables
83 # -----
84 # Micropolar elastic material parameters -- conversion
85 lamda = G*((2.*nu)/(1.-2.*nu))
86 mu     = G*((1.-2.*N**2)/(1.-N**2))
87 kappa  = G*((2.*N**2)/(1.-N**2))
88 gamma  = 4.*G*I**2
89
90 # Micropolar strain & micro-curvature -----
91 def epsilon(u, theta):
92
93     strain = as_tensor([[ u[0].dx(0),          u[1].dx(0) - theta ],
94                          [ u[0].dx(1) + theta, u[1].dx(1)          ]])
95
96     return strain
97
98
99 def epsilon_sym(u):
100
101     strain_sym = as_tensor([[ u[0].dx(0),
102                               (1./2.)*(u[0].dx(1) + u[1].dx(0)) ],
103                              [ (1./2.)*(u[0].dx(1) + u[1].dx(0)),
104                                u[1].dx(1)                          ]])
105
106     return strain_sym
107
108
109 def epsilon_skew(u, theta):
110
111     strain_skew = as_tensor([[ 0.0,
112                                (1./2.)*(u[1].dx(0) - u[0].dx(1)) - theta ],
113                               [ (1./2.)*(u[0].dx(1) - u[1].dx(0)) + theta,
114                                 0.0                                     ]])
115
116     return strain_skew
117
118
119 def phi(theta):
120
121     curvature = as_vector([ theta.dx(0),
122                             theta.dx(1) ])
123
124     return curvature
125 # -----

```

```

125
126
127 # Force stress & couple stress -----
128 def sigma_B(u):
129
130     eps_sym = epsilon_sym(u)
131
132     stress_B = lamda*tr(eps_sym)*Identity(2) + (2.*mu+kappa)*eps_sym
133
134     return stress_B
135
136
137 def sigma_C(u, theta):
138
139     eps_skew = epsilon_skew(u, theta)
140
141     stress_C = kappa*eps_skew
142
143     return stress_C
144
145
146 def sigma(u, theta):
147
148     stress = sigma_B(u) + sigma_C(u, theta)
149
150     return stress
151
152
153 def E3_sigma_C(u, theta):
154
155     stress_C = sigma_C(u, theta)
156
157     return stress_C[0,1] - stress_C[1,0]
158
159
160 def m_R(theta):
161
162     curvature = phi(theta)
163
164     couple = gamma*curvature
165
166     return couple
167 # -----
168
169 # -----
170 # Define variational form
171 # -----
172 # Define test & trial spaces
173 u, theta = TrialFunctions(V)
174 eta, zeta = TestFunctions(V)
175
176 # Bilinear form
177 a = inner(epsilon(eta,zeta), sigma(u, theta)) * dx \
178     + inner(phi(zeta), m_R(theta)) * dx \
179     #- inner(zeta, E3_sigma_C(u, theta)) * dx
180
181 L = dot(eta, traction) * ds(1)
182
183 # Solution
184 x_h = Function(V)
185
186 # Define variational problem & solver
187 problem = LinearVariationalProblem(a, L, x_h, BC)

```

```

188 solver = LinearVariationalSolver(problem)
189
190 # -----
191 # Solve system
192 # -----
193 solver.solve()
194 u_h, theta_h = x_h.split()
195
196 # -----
197 # Output results
198 # -----
199 # Field variables
200 vtkfile_u = File('./'+file_name+'/'+file_name+'_u.pvd')
201 vtkfile_u << u_h
202
203 vtkfile_theta = File('./'+file_name+'/'+file_name+'_theta.pvd')
204 vtkfile_theta << theta_h
205
206 # --- Stress output ---
207 # Define the stress tensor function space (Tensor element of degree 1,
208 # or higher if needed)
209 W = TensorFunctionSpace(mesh, "DG", 1) #Using DG space for discontinuous stress
210
211 # Calculate stress components
212 sigma_tensor = sigma(u_h, theta_h)
213 sigma_11 = project(sigma_tensor[0, 0]/trac, FunctionSpace(mesh, "DG", 1))
214 sigma_12 = project(sigma_tensor[0, 1]/trac, FunctionSpace(mesh, "DG", 1))
215 sigma_21 = project(sigma_tensor[1, 0]/trac, FunctionSpace(mesh, "DG", 1))
216 sigma_22 = project(sigma_tensor[1, 1]/trac, FunctionSpace(mesh, "DG", 1))
217
218 # Output the stress components to VTK files
219 vtkfile_sigma_11 = File('./' + file_name + '/' + file_name + '_n_11.pvd')
220 vtkfile_sigma_11 << sigma_11
221 vtkfile_sigma_12 = File('./' + file_name + '/' + file_name + '_n_12.pvd')
222 vtkfile_sigma_12 << sigma_12
223 vtkfile_sigma_21 = File('./' + file_name + '/' + file_name + '_n_21.pvd')
224 vtkfile_sigma_21 << sigma_21
225 vtkfile_sigma_22 = File('./' + file_name + '/' + file_name + '_n_22.pvd')
226 vtkfile_sigma_22 << sigma_22
227
228 # --- Von Mises stress output (optional) ---
229 # Define a function to calculate von Mises stress
230 def von_mises(sigma):
231     s11 = sigma[0,0]
232     s22 = sigma[1,1]
233     s12 = sigma[0,1]
234     return sqrt(0.5*((s11-s22)**2 + 6*s12**2) + s11**2 + s22**2)
235
236 von_mises_stress = von_mises(sigma_tensor)
237 Q = FunctionSpace(mesh, "DG", 1) # Scalar function space
238 von_mises_h = project(von_mises_stress, Q)
239
240 # Output von Mises stress to a VTK file
241 vtkfile_von_mises = File('./'+file_name+'/'+file_name+'_von_mises.pvd')
242 vtkfile_von_mises << von_mises_h
243
244 toc = time.time() - tic
245
246 print('Elapsed CPU time: ', toc, '[sec]')

```

Listing 1: FEniCS code for micropolar elasticity.

Appendix 2: Matlab Code for Analytical Verification

```
1 % Corrected Parameters (FEniCS paper alignment)
2 T = 1e6;           % Applied tension [Pa]
3 R = 1.0;           % Hole radius [m]
4 G = 50e9;          % Shear modulus [Pa]
5 nu = 0.3;          % Poisson's ratio
6 l = 2.0;           % Characteristic length = D (hole diameter) = 2R
7 N = 0.75;          % Coupling number
8 % Derived Micropolar Parameters
9 mu = G;
10 kappa = (2*mu*N^2)/(1 - N^2);
11 gamma = 4*mu*l^2; % Critical correction: l = 2R
12 % Eringen's Dimensionless Parameters
13 c = sqrt(gamma*(mu + kappa)/(kappa*(2*mu + kappa)));
14 b = sqrt(gamma/(2*(2*mu + kappa)));
15 % Stress Concentration Factor Formula
16 F1 = 8*(1-nu)*(b^2/c^2) / (4 + (R^2/c^2) + 2*(R/c)*(besselk(0,R/c)/besselk(1,R/c)));
17 Kt = (3 + F1)/(1 + F1);
18 fprintf('Corrected Stress Concentration Factor: K_t = %.4f\n', Kt);
```

Listing 2: MATLAB code for calculating stress concentration factor.